Instruction Set

What is an Instruction Set?

An **Instruction Set** is the complete set of instructions that a CPU can execute. For an **accumulator-based CPU**, all operations involve the **Accumulator (AC)** by default. Such CPUs typically:

- Use a *single-address format*, where only one memory address is specified per instruction
- Rely on the Accumulator (AC) as one of the operands for all operations

Instruction Set Table Breakdown

Type	Instruction	HDL Format	Assembly Format
Data	Load	AC := M(X)	LD X
	Store	M(X) := AC	ST X
Register Move	Move Register	DR := AC	MOV DR, AC
Arithmetic	Add	AC := AC + DR	ADD
	Subtract	AC := AC - DR	SUB
	And	AC := AC `AND` DR	AND
	Not	AC := `NOT` AC	NOT
Control	Branch	PC := M(adr)	BRA adr
	Branch if Zero	if AC = 0 then PC := M(adr)	BZ adr

Arithmetic Operation: Negation ($-X$)

Negation (computing $-X$) is achieved without a direct `NEG` instruction. It is implemented using subtraction logic.

Breakdown of Negation

Assume the Accumulator contains value X. To compute $-X$, perform the following:

HDL Format	Assembly Format	Explanation
DR := AC	MOV DR, AC	Save X into DR for later use
AC := AC - DR	SUB	AC = X - X = 0
AC := AC - DR	SUB	AC = 0 - X = $-X$

- The first `SUB` sets the accumulator to zero.
- The second `SUB` subtracts X again to result in $-X$.

This is how negation is implemented using basic subtraction in a minimal instruction set.

Summary

- The accumulator-based CPU operates with a simple but powerful instruction set.
- All operations center around manipulating the Accumulator and optionally the Data Register (DR).
- Even operations like *negation* can be constructed from basic instructions.
- This approach keeps hardware and instruction decoding simpler and more efficient.